

过滤器 *Filter* + 拦截器 *Interceptor* 综合案例

本案例实现 过滤器做统一限流 + 请求日志、拦截器做登录认证 + 权限校验的完整实战。

一、项目基础环境搭建

1. 新建 Spring Boot 项目

依赖选择：

- Spring Web
- Lombok

2. 项目结构

```
1  src/main/java/com/example/filterinterceptor/
2  |—— config/          # 配置类
3  |   |—— FilterConfig.java      # 过滤器注册配置
4  |   |—— InterceptorConfig.java # 拦截器注册配置
5  |—— filter/          # 过滤器
6  |   |—— RateLimitFilter.java   # 限流过滤器
7  |—— interceptor/     # 拦截器
8  |   |—— AuthInterceptor.java   # 认证权限拦截器
9  |—— controller/     # 测试接口
10 |   |—— TestController.java
11 |—— util/            # 工具类
12 |   |—— JwtUtil.java # JWT工具类（模拟登录认证）
13 |—— FilterInterceptorApplication.java # 启动类
```

二、过滤器 *Filter* 实战：接口限流 + 请求日志

过滤器作用：基于 Servlet 规范，在请求进入 Controller 之前 / 响应返回之后执行，适合全局请求处理、限流、跨域、日志等。

本次实现：IP 接口限流（1 分钟内最多访问 10 次）+ 打印请求日志。

1. 限流过滤器实现 RateLimitFilter.java

```

1  package com.example.filterinterceptor.filter;
2
3  import jakarta.servlet.*;
4  import jakarta.servlet.http.HttpServletRequest;
5
6  import jakarta.servlet.http.HttpServletResponse;
7  import lombok.extern.slf4j.Slf4j;
8  import org.springframework.http.HttpStatus;
9
10 import java.io.IOException;
11 import java.util.Map;
12 import java.util.concurrent.ConcurrentHashMap;
13 import java.util.concurrent.atomic.AtomicInteger;
14
15 /**
16  * 过滤器: IP限流 + 请求日志打印
17  * 规则: 同一个IP 1分钟内最多访问10次
18  */
19 @Slf4j
20 public class RateLimitFilter implements Filter {
21
22     // 存储IP访问次数 (线程安全)
23     private static final Map<String, AtomicInteger> IP_COUNT_MAP = new
24     ConcurrentHashMap<>();
25
26     // 最大访问次数
27     private static final int MAX_COUNT = 10;
28
29     // 时间窗口: 1分钟 (毫秒)
30     private static final long WINDOW_TIME = 60 * 1000;
31
32     // 初始化: 项目启动时执行一次
33     @Override
34     public void init(FilterConfig filterConfig) throws ServletException {
35         log.info("限流过滤器初始化完成");
36         // 定时清空Map (防止内存溢出)
37         new Thread(() -> {
38             while (true) {
39                 try {
40                     Thread.sleep(WINDOW_TIME);
41                     IP_COUNT_MAP.clear();
42                 } catch (InterruptedException e) {
43                     e.printStackTrace();
44                 }
45             }
46         }).start();
47     }
48
49     // 核心过滤逻辑
50     @Override

```

```

47     public void doFilter(ServletRequest request, ServletResponse response,
    FilterChain chain)
48         throws IOException, ServletException {
49         HttpServletRequest req = (HttpServletRequest) request;
50         HttpServletResponse resp = (HttpServletResponse) response;
51
52         // 1. 获取请求IP
53         String ip = req.getRemoteAddr();
54         String uri = req.getRequestURI();
55         String method = req.getMethod();
56
57         // 2. 打印请求日志
58         log.info("【过滤器】请求日志 -> IP:{}, 请求方式:{}, 请求路径:{}, ip, method,
    uri);
59
60         // 3. 限流逻辑
61         AtomicInteger count = IP_COUNT_MAP.getOrDefault(ip, new
    AtomicInteger(0));
62         int currentCount = count.incrementAndGet();
63         IP_COUNT_MAP.put(ip, count);
64
65         // 超过最大次数, 直接返回错误
66         if (currentCount > MAX_COUNT) {
67             log.warn("【过滤器】IP:{} 访问频率过高, 已限流", ip);
68             resp.setContentType("application/json;charset=utf-8");
69             resp.getWriter().write("{\"code\":429,\"msg\":\"请求过于频繁, 请1分钟后再
    试\"}");
70             resp.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
71             return;
72         }
73
74         // 4. 放行请求 (进入下一个过滤器/Controller)
75         chain.doFilter(request, response);
76
77         // 5. 响应返回后执行 (后置处理)
78         log.info("【过滤器】响应已返回 -> IP:{}, 路径:{}, ip, uri);
79     }
80
81     // 销毁: 项目停止时执行
82     @Override
83     public void destroy() {
84         log.info("限流过滤器已销毁");
85     }
86 }

```

2. 过滤器注册配置 FilterConfig.java

SpringBoot 3.x 推荐注册 Bean 方式配置过滤器（不使用 @WebFilter）：

```
1 package com.example.filterinterceptor.config;
2
3 import com.example.filterinterceptor.filter.RateLimitFilter;
4 import org.springframework.boot.web.servlet.FilterRegistrationBean;
5 import org.springframework.context.annotation.Bean;
6 import org.springframework.context.annotation.Configuration;
7
8 /**
9  * 过滤器配置类：注册自定义过滤器
10  */
11 @Configuration
12 public class FilterConfig {
13
14     @Bean
15     public FilterRegistrationBean<RateLimitFilter> rateLimitFilterRegistration()
16     {
17         FilterRegistrationBean<RateLimitFilter> registrationBean = new
18         FilterRegistrationBean<>();
19         registrationBean.setFilter(new RateLimitFilter());
20         // 过滤请求
21         registrationBean.addUrlPatterns("/api/hello");
22         registrationBean.setOrder(1);
23         registrationBean.setName("rateLimitFilter");
24         return registrationBean;
25     }
26 }
```

三、拦截器 *Interceptor* 实战：登录认证 + 权限校验

拦截器作用：基于 Spring MVC，在 Controller 方法执行前后 / 视图渲染后执行，能获取 Controller 方法信息，适合登录认证、权限校验、日志埋点。

本次实现：

1. 登录接口放行，其他接口必须携带有效 Token
2. 管理员接口校验权限

1. JWT 工具类（模拟登录认证）JwtUtil.java

```
1 package com.example.filterinterceptor.util;
2
3 import java.util.Date;
```

```

3  import io.jsonwebtoken.Claims;
4  import io.jsonwebtoken.Jwts;
5  import io.jsonwebtoken.security.Keys;
6  import org.springframework.stereotype.Component;
7
8  import javax.crypto.SecretKey;
9  import java.util.Date;
10 import java.util.HashMap;
11 import java.util.Map;
12
13 /**
14  * JWT工具类：生成/解析Token（模拟登录认证）
15  */
16 @Component
17 public class JwtUtil {
18     // 密钥（实际项目放在配置文件）
19     private static final String SECRET_KEY =
20 "abcdefghijklmnopqrstuvwxyz01234567890abcdefghijklmnopqrstuvwxyz";
21
22     // Token过期时间：1小时
23     private static final long EXPIRATION_TIME = 60 * 60 * 1000L;
24
25     // 获取密钥
26     private SecretKey getSecretKey() {
27         return Keys.hmacShaKeyFor(SECRET_KEY.getBytes());
28     }
29
30     /**
31     * 生成Token
32     *
33     * @param username 用户名
34     * @param role     角色：admin/user
35     * @return token
36     */
37     public String generateToken(String username, String role) {
38         Map<String, Object> claims = new HashMap<>();
39         claims.put("username", username);
40         claims.put("role", role);
41
42         return Jwts.builder()
43             .claims(claims)
44             .issuedAt(new Date())
45             .expiration(new Date(System.currentTimeMillis() +
46 EXPIRATION_TIME))
47             .signWith(getSecretKey())
48             .compact();
49     }
50
51     /**

```

```

49     * 解析Token
50     */
51     public Claims parseToken(String token) {
52         try {
53             return Jwts.parser()
54                 .verifyWith(getSecretKey())
55                 .build()
56                 .parseSignedClaims(token)
57                 .getPayload();
58         } catch (Exception e) {
59             return null;
60         }
61     }
62 }

```

2. 拦截器核心实现 AuthInterceptor.java

```

1  import io.jsonwebtoken.Claims;
2  import jakarta.servlet.http.HttpServletRequest;
3  import jakarta.servlet.http.HttpServletResponse;
4  import lombok.RequiredArgsConstructor;
5
6  import lombok.extern.slf4j.Slf4j;
7  import org.springframework.stereotype.Component;
8  import org.springframework.web.servlet.HandlerInterceptor;
9  import org.springframework.web.servlet.ModelAndView;
10 import top.mqxu.week09.utils.JwtUtil;
11
12 /**
13  * 拦截器：登录认证 + 权限校验
14  */
15 @Component
16 @Slf4j
17 @RequiredArgsConstructor
18 public class AuthInterceptor implements HandlerInterceptor {
19     private final JwtUtil jwtUtil;
20
21     /**
22      * Controller方法执行前执行（核心逻辑）
23      *
24      * @return true: 放行 false: 拦截
25      */
26     @Override
27     public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler)
28         throws Exception {

```

```

29     String uri = request.getRequestURI();
30     log.info("【拦截器】进入请求: {}", uri);
31
32     // 1. 放行登录接口
33     if ("/api/login".equals(uri)) {
34         return true;
35     }
36
37     // 2. 获取请求头Token
38     String token = request.getHeader("token");
39     if (token == null || token.isEmpty()) {
40         log.warn("【拦截器】未登录, 请求被拦截: {}", uri);
41         response.setContentType("application/json;charset=utf-8");
42         response.getWriter().write("{\"code\":401,\"msg\":\"请先登录\"}");
43         return false;
44     }
45
46     // 3. 解析Token
47     Claims claims = jwtUtil.parseToken(token);
48     if (claims == null) {
49         log.warn("【拦截器】Token无效/已过期: {}", uri);
50         response.setContentType("application/json;charset=utf-8");
51         response.getWriter().write("{\"code\":401,\"msg\":\"登录已过期, 请重新登
录\"}");
52         return false;
53     }
54
55     // 4. 管理员接口权限校验
56     String role = claims.get("role", String.class);
57     if (uri.startsWith("/api/admin") && !"admin".equals(role)) {
58         log.warn("【拦截器】权限不足, 禁止访问管理员接口: {}", uri);
59         response.setContentType("application/json;charset=utf-8");
60         response.getWriter().write("{\"code\":403,\"msg\":\"权限不足, 无法访问
\"}");
61         return false;
62     }
63
64     // 5. 把用户信息存入request, 供Controller使用
65     request.setAttribute("username", claims.get("username"));
66     request.setAttribute("role", role);
67     log.info("【拦截器】认证通过, 用户: {}, 角色: {}", claims.get("username"),
role);
68     return true;
69 }
70
71 /**
72  * Controller方法执行后, 视图渲染前执行
73  */

```

```

74     @Override
75     public void postHandle(HttpServletRequest request, HttpServletResponse
response, Object handler,
76                             ModelAndView modelAndView) throws Exception {
77         log.info("【拦截器】Controller执行完成");
78     }
79
80     /**
81      * 视图渲染完成后执行（最后执行）
82      */
83     @Override
84     public void afterCompletion(HttpServletRequest request, HttpServletResponse
response, Object handler,
85                                 Exception ex) throws Exception {
86         log.info("【拦截器】请求完全处理完毕");
87     }
88 }
89

```

3. 拦截器注册配置 InterceptorConfig.java

```

1  @Configuration
2  @RequiredArgsConstructor
3  public class InterceptorConfig implements WebMvcConfigurer {
4
5      private final AuthInterceptor authInterceptor;
6
7      @Override
8      public void addInterceptors(InterceptorRegistry registry) {
9          registry.addInterceptor(authInterceptor)
10
11              .addPathPatterns("/api/**")    // 添加需要拦截的接口
12              .excludePathPatterns("/api/login") // 添加放行的接口
13              .excludePathPatterns("/api/hello") // 添加放行的接口
14              .order(2);
15      }
16  }

```

四、测试接口 Controller

```

1  import jakarta.servlet.http.HttpServletRequest;
2  import lombok.RequiredArgsConstructor;
3  import org.springframework.web.bind.annotation.GetMapping;
4  import org.springframework.web.bind.annotation.RequestMapping;

```



```
4 import org.springframework.web.bind.annotation.RequestMapping;
5 import org.springframework.web.bind.annotation.RestController;
6 import top.mqxu.week09.utils.JwtUtil;
7
8 @RestController
9 @RequestMapping("/api")
10 @RequiredArgsConstructor
11 public class TestController {
12
13     private final JwtUtil jwtUtil;
14
15     // 登录接口 (放行)
16     @GetMapping("/login")
17     public String login(String username, String password) {
18         // 模拟登录校验
19         if ("admin".equals(username) && "123456".equals(password)) {
20             String token = jwtUtil.generateToken(username, "admin");
21             return "登录成功! 管理员Token: " + token;
22         }
23         if ("user".equals(username) && "123456".equals(password)) {
24             String token = jwtUtil.generateToken(username, "user");
25             return "登录成功! 普通用户Token: " + token;
26         }
27         return "用户名或密码错误";
28     }
29
30     // 普通接口 (需要登录)
31     @GetMapping("/user/info")
32     public String userInfo(HttpServletRequest request) {
33         String username = (String) request.getAttribute("username");
34         String role = (String) request.getAttribute("role");
35         return "当前登录用户: " + username + ", 角色: " + role;
36     }
37
38     // 管理员接口 (需要管理员权限)
39     @GetMapping("/admin/list")
40
41     public String adminList() {
42         return "管理员数据: 用户列表、系统配置";
43     }
44
45     // 测试限流接口
46     @GetMapping("/test/rate")
47     public String testRate() {
48         return "接口访问成功";
49     }
50 }
```

五、启动项目 + 完整测试流程

1. 启动项目

控制台会打印：限流过滤器初始化完成

2. 测试 1：限流功能（过滤器）

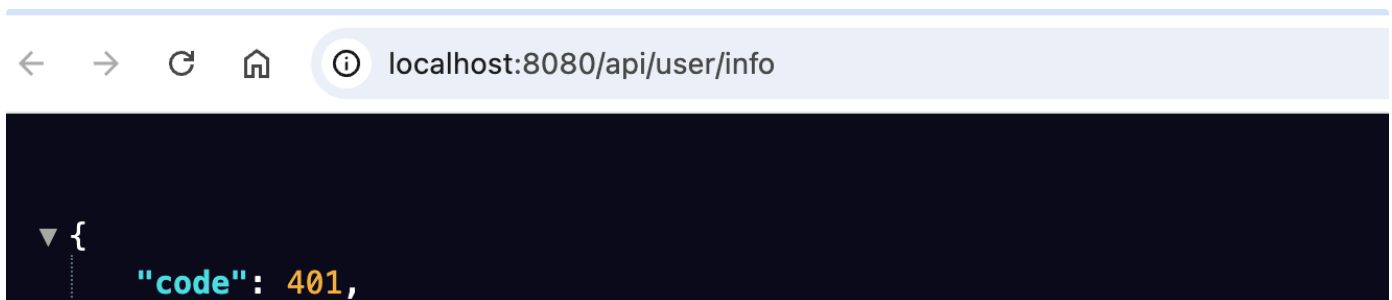
访问：<http://localhost:8080/api/hello>

- 前 10 次正常返回
- 第 11 次返回：



3. 测试 2：登录认证（拦截器）

1. 未登录访问：<http://localhost:8080/api/user/info> → 返回 请先登录



```
}  
"msg": "请先登录"
```

1. 登录获取 Token：

- 管理员：<http://localhost:8080/api/login?username=admin&password=123456>

localhost:8080/api/login?username=admin&password=123456

登录成功！管理员Token：
eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoieWRTaW4iLCJ1c2VybmFtZSI6ImFkbWluliwiaWF0IjoxNzc3NTM5MTQ5LjE3NDI3NDI5.yiBCLAC4t18hCwOhJrrH6W5cQenabZgFm9JALs0yNil

- 普通用户：<http://localhost:8080/api/login?username=user&password=123456>

localhost:8080/api/login?username=user&password=123456

登录成功！普通用户Token：eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiaXNlciIsInVzZXJuYXV1IjoiaXNlciIsImh0bGVzZXNzUzOTE3MSwiZXhwIjoxNzc3NTQyNzc3Fm8sLhyBeAZPD4ne4TFScobVBJZ4-pUN-9dtuXLO5ll

4. 测试 3：权限校验（拦截器）

- 普通用户携带请求头 token 访问 <http://localhost:8080/api/admin/list> → 权限不足

GET http://localhost:8080 /api/admin/list

发送

Params Body Headers 1 Cookies Auth 前置操作 后置操作 1 设置



- 管理员携带请求头 token 访问 <http://localhost:8080/api/admin/list> → 正常返回数据



六、过滤器 vs 拦截器 核心区别（面试重点）

对比项	过滤器 <i>Filter</i>	拦截器 <i>Interceptor</i>
规范	Servlet 规范	Spring MVC 规范
执行时机	请求进入 Spring 容器之前 / 响应返回之后	Controller 方法执行前后
依赖容器	依赖 Tomcat 等 Servlet 容器	依赖 Spring 容器
能获取的对象	Request/Response	Request/Response/Controller 方法信息
使用场景	全局限流、跨域、日志、字符编码过滤	登录认证、权限校验、方法级日志
终止方式	直接 return 响应	return false

七、依赖补充（pom.xml）

如果 JWT 包报错，手动引入：

```
1  <!-- JJWT 依赖（生成Token） -->
2  <dependency>
3      <groupId>io.jsonwebtoken</groupId>
4      <artifactId>jjwt-api</artifactId>
5      <version>0.12.6</version>
6  </dependency>
7  <dependency>
8      <groupId>io.jsonwebtoken</groupId>
9      <artifactId>jjwt-impl</artifactId>
10     <version>0.12.6</version>
11     <scope>runtime</scope>
12 </dependency>
13 <dependency>
14     <groupId>io.jsonwebtoken</groupId>
15     <artifactId>jjwt-jackson</artifactId>
16     <version>0.12.6</version>
17     <scope>runtime</scope>
18 </dependency>
```

总结

1. **过滤器**：适合全局请求预处理，本次实现了 IP 限流 + 请求日志，执行时机最早。
2. **拦截器**：适合业务层认证鉴权，本次实现了登录校验 + 权限控制，能获取 Spring 上下文。
3. 执行顺序：请求 → 过滤器 → 拦截器 → Controller → 拦截器 → 过滤器 → 响应。